

Encapsulation with Functions

Functions are a prime example of encapsulation because when you call a function, you generally don't care *how* the function works internally. A well-written function contains a series of steps that make up a larger single task, which you do care about. The name of the function should describe the action that its code embodies. Consider the built-in `len()` function from the Python Standard Library, used to find the number of characters in a string or elements in a list. You pass in a string or list, and it returns the count. When you write code that calls this function, you don't care how `len()` does what it does. You don't stop to think about whether the code of the function contains two lines or two thousand lines, whether it uses one local variable or a hundred. You just need to know what argument to pass in and how to use the result that's returned.

The same is true of functions that you write, like this function that calculates and returns the average of a list of numbers:

```
def calculateAverage(numbersList):
    total = 0.0
    for number in numbersList:
        total = total + number
    nElements = len(numbersList)
    average = total / nElements
    return average
```

Once you've tested a function like this and found that it works, you no longer have to worry about the details of the implementation. You only need to know what argument(s) to send into the function and what it returns.

However, if one day you find that there is a much simpler or faster algorithm to calculate an average, you could rewrite the function in a new way. As long as the interface (the inputs and outputs) does not change, there is no need to change any calls to the function. This type of modularization makes the code more maintainable.